Steps for GPU Training

1. Check GPU Availability

Before starting, verify if a GPU is available. If available, select it; otherwise, use the CPU.

Python

```
import torch

# Check for GPU
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Using device: {device}")
```

2. Move the Model to GPU

Move your model to the selected device (`cuda` for GPU or `cpu`) so that all computations occur on the same device.

Python

```
model = MyModel() # Replace with your model
model = model.to(device) # Move model to GPU
```

3. Modify the Training Loop by Moving Data to GPU

Ensure that each batch of data (features and labels) is moved to the GPU before processing. This ensures that both the model and data are on the same device.

Python

```
for batch_features, batch_labels in train_loader:
    # Move data to GPU
    batch_features, batch_labels = batch_features.to(device), batch_labels.to(device)
```

4. Modify the Evaluation Loop by Moving Data to GPU

Similarly, ensure test data is moved to the GPU during evaluation. Disable gradient calculations using `torch.no_grad()` for efficiency.

Python

```
with torch.no_grad():
    for batch_features, batch_labels in test_loader:
        # Move data to GPU
        batch_features, batch_labels = batch_features.to(device), batch_labels.to(device)
```

5. Optimize the GPU Usage

To make the best use of GPU resources, apply the following optimizations:

a. Use Larger Batch Sizes

Larger batch sizes can better utilize GPU memory and reduce computation time per epoch (if memory allows).

Python

```
train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False, pin_memory=True)
```

b. Enable DataLoader Pinning

Use `pin_memory=True` in `DataLoader` to speed up data transfer from CPU to GPU.

Python

```
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True, pin_memory=True)
```